

FloodLite: A Multi-Platform Edge-Deployment Benchmark for Lightweight Flood Segmentation

Md Ibrahim Khalil^{a,*}

^a*Sensa AS, Oslo, Norway*

Abstract

Operational flood-extent mapping is bottlenecked not by segmentation accuracy on academic benchmarks but by the gap between published models and the constrained hardware that humanitarian responders actually deploy. Across the recent flood-segmentation literature, every paper reports IoU on a benchmark dataset; none reports the per-platform inference latency, post-quantization accuracy, or model size that determines whether a model can run on a tablet, in a browser, or on an embedded ARM device. We close that gap. *FloodLite* is a systematic edge-deployment benchmark for lightweight flood segmentation built around three UNet students — MobileNetV3-Small, EfficientNet-Lite0, and MobileViT-XXS — and a UNet+EfficientNet-B0 teacher, evaluated under three-fold cross-validation on the Flood Semantic Segmentation Dataset (FSSD). We measure FP32 and INT8 inference latency on Apple M2 Pro (ONNX Runtime CPU) and in-browser WebAssembly SIMD (ONNX Runtime Web), and we report a paired-Wilcoxon-tested ablation of response-only, feature-only, and combined response+feature knowledge distillation (KD). We find that (i) on this saturated benchmark, ImageNet-

*Corresponding author.

Email address: `ibrahim@sensa.no` (Md Ibrahim Khalil)

pretrained lightweight students match or exceed the teacher without any distillation — the best student, MobileViT-XXS, attains $\text{IoU} = 0.9367 \pm 0.0055$ against the teacher’s 0.9257 ± 0.0078 ; (ii) combined response+feature KD never improves over the no-KD baseline on any of the three architectures and degrades MobileViT-XXS by 0.005 IoU points; (iii) post-training INT8 quantization stability is sharply architecture-dependent — EfficientNet-Lite0 loses only 0.043 IoU points after INT8 conversion and reaches 58 FPS on M2 Pro CPU, whereas MobileNetV3-Small (HardSwish activations) and MobileViT-XXS (LayerNorm + self-attention) suffer 0.45–0.46 IoU collapses and high cross-fold variance under the same static QDQ recipe. The recommended deployable configuration — EfficientNet-Lite0-UNet INT8 served by ONNX Runtime CPU — runs at 58 FPS on a 2023 laptop CPU and 6 FPS in a single-threaded WebAssembly browser sandbox at $3.83\times$ size reduction, with no measurable accuracy loss versus the teacher. All trained models, ONNX exports, benchmarking scripts, and a Zenodo-archived release are made publicly available.

Keywords: flood segmentation, edge deployment, knowledge distillation, post-training quantization, ONNX Runtime, WebAssembly, remote sensing, lightweight neural networks, disaster response

1. Introduction

Floods are the most frequent and economically damaging natural disasters globally, accounting for 38.7% of natural disasters and affecting 43% of disaster-affected populations between 2000 and 2009 [1]. Rapid urbanisation continues to amplify exposure [2]. Because traditional hydrological and

6 hydraulic models suffer from high computational demand, coarse spatial res-
7 olution, and limited real-time adaptability [3], deep-learning segmentation
8 has emerged as a leading approach to automated flood-extent delineation
9 from satellite, drone, and ground imagery [4].

10 Recent benchmark studies in this space have converged on encoder–decoder
11 architectures — UNet [5], SegNet [6], and DeepLabV3+ [7] — paired with
12 strong feature-extraction backbones, most commonly EfficientNet [8] and
13 ResNet [9]. Karcı *et al.* [10] provide the most comprehensive systematic com-
14 parison of these combinations on the public Flood Semantic Segmentation
15 Dataset (FSSD), reporting that UNet + EfficientNet achieves the highest seg-
16 mentation performance (IoU 0.927, F1 0.957, accuracy 0.971) and conclude
17 that this combination “is ideal for near real-time operational deployment.”

18 A careful reading of the broader flood-segmentation literature, however,
19 reveals that this operational-deployment claim has not actually been eval-
20 uated. Across the 26 related works surveyed by Karcı *et al.* [10] and adja-
21 cent studies on UAV [11], SAR [12], and multi-modal flood mapping [13],
22 every paper reports segmentation accuracy on academic benchmarks; *none*
23 *reports model parameters, FLOPs, per-platform inference latency, or post-*
24 *quantization accuracy.* This omission is consequential. UNet + EfficientNet-
25 B7 — the configuration favoured in [10] — has approximately 63 M paramet-
26 ers and 50 GFLOPs at 256×256 input resolution. Such a model cannot run
27 on the drones, embedded sensors, Raspberry-Pi-class community stations, or
28 offline tablet apps that humanitarian organisations such as UN-SPIDER and
29 the ICRC actually deploy in flood zones, where reliable internet connectivity
30 is the exception rather than the rule [14].

31 *1.1. Contributions*

32 This paper makes three contributions.

- 33 **1. First systematic multi-platform edge-deployment benchmark**
34 **for flood segmentation.** We report FP32 and INT8 inference latency
35 for three lightweight UNet students on two deployment targets — Apple
36 M2 Pro via ONNX Runtime CPU, and browser WebAssembly SIMD
37 via ONNX Runtime Web 1.17 — together with per-model parameter
38 counts, ONNX disk size, and FP32-to-INT8 size-reduction factors. ONNX
39 serves as the deployment hub: every measured latency reflects the same
40 exported artifact running under a different execution provider.
- 41 **2. Negative empirical finding on knowledge distillation for FSSD-**
42 **class flood segmentation.** Across three architectures $\times$ four KD con-
43 figurations $\times$ three folds, no KD configuration improves on the no-KD
44 baseline. ImageNet-pretrained lightweight students match or exceed
45 the UNet+EfficientNet-B0 teacher without distillation, and combined
46 response + feature KD strictly underperforms the no-KD baseline on
47 every student. We provide a mechanistic explanation (saturated 663-
48 image benchmark; strong ImageNet priors leave no headroom for the
49 teacher’s soft targets) and report exact paired-Wilcoxon p-values, with
50 explicit caveats about the limited statistical power of three folds.
- 51 **3. Architecture-dependent post-training quantization stability char-**
52 **acterisation.** Under an identical ONNX static QDQ INT8 recipe (Per-
53 centile 99.999% calibration, per-tensor activations, per-channel weights
54 restricted to Conv/Gemm/MatMul), EfficientNet-Lite0 quantizes cleanly
55 ($\Delta\text{IoU} = -0.043 \pm 0.016$) while MobileNetV3-Small and MobileViT-

XXS suffer 0.45–0.46 IoU collapses with cross-fold standard deviations of 0.19–0.34. We trace the failure modes to (i) the HardSwish activations in MobileNetV3-Small that lack bounded calibration ranges, and (ii) the LayerNorm and self-attention blocks in MobileViT-XXS that produce rank-deficient calibration statistics under per-channel quantization. The implication for practitioners: *bounded-activation backbones (ReLU6) post-training-quantize reliably for flood segmentation; HardSwish and attention-based backbones require quantization-aware training (QAT).*

We make our trained checkpoints, the ONNX FP32 and INT8 exports, the per-platform benchmarking harnesses, and a Zenodo-archived release of the full code repository publicly available.

1.2. Scope and pivot from initial framing

This work was originally framed as a knowledge-distillation recipe for compressing a heavy flood-segmentation teacher. The empirical results did not support that framing, and we report the negative-KD finding rather than search for a configuration that would justify the original framing. The contribution pivots to the per-platform edge benchmark and the architecture-dependent PTQ characterisation. We hope this orientation is more useful to practitioners than another paper-grade KD recipe, and that the negative result helps future researchers avoid investing compute in distillation on saturated benchmarks where strong ImageNet pretraining has already closed the gap.

The remainder of the paper is organised as follows. Section 2 reviews

80 relevant prior work on flood segmentation, lightweight segmentation back-
81 bones, knowledge distillation, and post-training quantization. Section 3 de-
82 tails the FloodLite methodology. Section 4 describes the experimental setup,
83 including the 3-fold protocol and per-platform latency rigging. Section 5
84 reports results across Tables 1–5 and Figures 1–2. Section 6 discusses the
85 negative-KD finding, per-platform deployment recommendations, and the
86 architecture-dependent PTQ insight. Section 7 lists limitations. Section 8
87 concludes.

88 2. Related Work

89 2.1. Deep Learning for Flood Segmentation

90 Convolutional encoder–decoder architectures dominate the deep-learning
91 flood-segmentation literature. Pech-May *et al.* [12] applied U-Net to Sentinel-
92 1 SAR imagery for flooded-area detection. Bahrami and Arbabkhah [15]
93 compared SegNet, UNet, and FCN32 for floodplain segmentation, report-
94 ing SegNet’s max-pooling indices as advantageous on smooth water-surface
95 transitions. Patil *et al.* [16] used an ensemble of ResNet34, InceptionV3, and
96 VGG16 on Sentinel-1 SAR for flood-damage assessment, reaching an IoU of
97 0.758. Ghosh *et al.* [17] used a Nested U-Net with FPN and EfficientNet-
98 B7 backbone on the NASA IEEE GRSS benchmark, reporting that mod-
99 els trained on specific events generalise to new geographies. Chamatidis *et*
100 *al.* [13] introduced Vision Transformers (ViT) to the task and reported a 9–
101 15% improvement over comparable CNN baselines on Sentinel-1 and Sentinel-
102 2 imagery. Most recently, Karci *et al.* [10] performed the most comprehensive
103 systematic comparison to date, evaluating UNet, SegNet, and DeepLabV3+

each with EfficientNet and ResNet backbones, and concluded that UNet + EfficientNet is the strongest configuration on FSSD.

Across this entire body of work, model efficiency is essentially absent from the evaluation. Hernández *et al.* [11] gesture toward edge deployment of UAV-based flood detection, but retain heavy backbones and report no parameter, FLOP, or per-platform latency numbers. To the best of our knowledge, no prior flood-segmentation paper has measured on-device inference latency on Apple Silicon or in a browser WebAssembly sandbox, nor has any prior work quantified the per-architecture stability of INT8 post-training quantization for this task.

2.2. Knowledge Distillation for Semantic Segmentation

Knowledge distillation (KD), introduced by Hinton *et al.* [18], trains a small student network to match the soft outputs of a larger teacher. Romero *et al.* [19] extended this to feature-space alignment with FitNets, training the student to match intermediate feature representations through a learned adapter. For dense-prediction tasks, Liu *et al.* [20] proposed structured-knowledge distillation (SKD) that aligns pairwise pixel similarities; Wang *et al.* [21] proposed inter- and intra-class distance distillation specifically for segmentation; and He *et al.* [22] introduced channel-wise KD for compact segmentation models. These methods routinely achieve compression ratios of 5–20 $\times$ with sub-1% mIoU degradation on Cityscapes and ADE20K. To our knowledge, no prior work has applied KD to flood segmentation.

Two findings from the broader KD literature are particularly relevant here. First, KD gains shrink sharply as student capacity and pretraining quality grow [19, 23]. Second, on saturated benchmarks where the student

129 matches or exceeds the teacher without distillation, the response-based term
 130 reduces to a noise-injection regulariser rather than a genuine transfer sig-
 131 nal. Our empirical results on FSSD (Section 5.2) are consistent with both
 132 observations.

133 2.3. *Lightweight Backbones for Mobile Vision*

134 MobileNetV3 [24] uses neural architecture search and the H-swish ac-
 135 tivation to produce CPU-friendly classification networks; the Small variant
 136 has approximately 2.5 M parameters. EfficientNet-Lite [8] is a quantization-
 137 friendly redesign of EfficientNet that removes squeeze-and-excitation and
 138 Swish activations to ease INT8 conversion, with the Lite0 variant containing
 139 approximately 4.7 M parameters. MobileViT [25] introduces hybrid CNN-
 140 Transformer blocks; the XXS variant has only 1.3 M parameters yet is com-
 141 petitive with much larger models on ImageNet. MobileNetV2 [26], included
 142 as an off-the-shelf baseline in this work, predates these but remains a strong
 143 reference point and exhibits the ReLU6-bounded activation profile we asso-
 144 ciate with PTQ stability (Section 5.3). We adapt all four as encoders in a
 145 U-Net-style decoder.

146 Beyond the four classification-derived backbones we evaluate, several
 147 segmentation-specific lightweight architectures have been proposed. BiSeN-
 148 etV2 [27] uses a two-pathway design (spatial + context) optimised for real-
 149 time urban-scene segmentation. PIDNet [28] reframes the three-pathway
 150 split as a proportional–integral–derivative controller. Fast-SCNN [29] targets
 151 sub-150-MB-FLOP inference at 1024×2048 . We do not evaluate these in this
 152 work because (a) they are designed for and benchmarked on Cityscapes-style
 153 urban scenes, not the heterogeneous viewpoints (aerial / oblique / ground)

154 of FSSD, and (b) our objective is to characterise the deployability of the
 155 *commodity* UNet + ImageNet-pretrained-backbone family that the flood-
 156 segmentation literature has already converged on, not to propose a new ar-
 157 chitecture.

158 2.4. Post-Training Quantization

159 Post-training quantization (PTQ) maps FP32 weights and activations to
 160 lower-precision integer representations [30, 31]. Two main flavours exist: dy-
 161 namic PTQ, which only quantizes weights and computes activation ranges
 162 at inference time, and static PTQ, which calibrates activation ranges offline
 163 against a representative dataset. For convolutional networks, the ONNX
 164 static QDQ (Quantize-DeQuantize) format with per-channel weight quant-
 165 ization and per-tensor activation quantization is the *de facto* standard [32]
 166 and is supported across major edge runtimes (ONNX Runtime CPU, Tensor-
 167 Flow Lite, CoreML). The empirical conventional wisdom — that “INT8 PTQ
 168 retains accuracy within 1 IoU point of FP32” [31] — is derived from classifica-
 169 tion networks with bounded-range activations (ReLU, ReLU6) on ImageNet-
 170 scale benchmarks. As we show in Section 5.3, this assumption does not hold
 171 uniformly across the lightweight architectures used in this study: HardSwish-
 172 based and attention-based backbones can suffer order-of-magnitude larger
 173 PTQ degradation on flood-segmentation pixels than the classification liter-
 174 ature would predict.

175 2.5. On-Device Disaster Response

176 Operational disaster-response platforms must function under intermittent
 177 connectivity, limited power, and heterogeneous hardware. Edge AI in human-

178 itarian contexts has been explored by Munawar *et al.* [14] and Kuglitsch *et*
 179 *al.* [33]. The deployment platforms we benchmark — Apple Silicon CPU
 180 and browser WebAssembly — correspond to two real operational profiles:
 181 field-laptop and UAV inference, and citizen-science web portals. Raspberry-
 182 Pi-class single-board computers, AWS Graviton ARM CPU, and Android
 183 edge targets are discussed in Section 7 as straightforward but unscheduled
 184 extensions.

185 3. Materials and Methods

186 3.1. Dataset

187 We use the public Flood Semantic Segmentation Dataset (FSSD) [34],
 188 which is also the evaluation dataset of Karcı *et al.* [10] and therefore en-
 189 ables direct head-to-head comparison. FSSD comprises 663 RGB images of
 190 flood-affected areas, each at 512×512px, paired with binary segmentation
 191 masks (flood = 1, non-flood = 0). Images capture flooded regions from vari-
 192 ous viewpoints (aerial, ground, oblique). We resize all images and masks
 193 to 256×256px and normalise pixel values to [0, 1] followed by ImageNet
 194 mean/std normalisation.

195 We adopt 3-fold cross-validation with a fixed random seed: `KFold(n_splits=5,`
 196 `shuffle=True, random_state=42)`, evaluated only on folds {0, 1, 2} of the
 197 5-fold partition. The reduction from 5 to 3 folds was a compute-budget
 198 decision driven by the negative-KD finding of Section 5.2: once distillation
 199 was shown to not improve over the no-KD baseline, the remaining experi-
 200 mental contrasts (per-architecture PTQ stability, per-platform latency) could
 201 be drawn from three folds with adequate variance estimation. We report all

202 results as mean $\pm$ standard deviation across the three folds, and explicitly
203 caveat all paired-Wilcoxon p-values for low statistical power (minimum re-
204 portable two-sided p-value at $n = 3$ is 0.25).

205 3.2. Teacher Model

206 Our teacher is a UNet decoder with EfficientNet-B0 encoder [8], initialised
207 from ImageNet weights. While Karçı *et al.* [10] use EfficientNet-B7 in their
208 best-performing configuration, we deliberately choose B0 as our teacher for
209 two reasons: first, B0 is significantly cheaper to train, fitting within free-tier
210 GPU budgets (Kaggle T4 and Google Colab); second, B0 is itself repres-
211 entative of the strongest configurations realistically used in practice, and a
212 successful distillation from B0 implies a successful distillation from any lar-
213 ger backbone in the same family. The teacher has approximately 6.25 M
214 parameters at 256×256 resolution.

215 3.3. Student Architectures and the MobileNetV2 Baseline

216 We evaluate three lightweight UNet student configurations together with
217 a MobileNetV2 [26] off-the-shelf baseline that controls for “is the teacher just
218 over-parameterised?”. Each student replaces the teacher’s EfficientNet-B0
219 encoder with a smaller backbone while keeping the UNet decoder topology
220 identical. All four configurations are initialised from ImageNet weights via
221 the `segmentation_models_pytorch` library [35] and `timm` [36].

222 Model footprints (parameter counts and FP32/INT8 ONNX disk sizes)
223 appear in Table 1.

Table 1: **Model footprints.** Parameter counts from `torchinfo.summary`; ONNX disk sizes from `os.path.getsize` on the exported `.onnx` files. FLOPs deferred to the camera-ready release.

Model	Params (M)	FP32 ONNX (MB)	INT8 ONNX (MB)
Teacher (UNet+EffB0)	6.25	—	—
MobileNetV2 baseline	6.63	—	—
MobileNetV3-Small	3.59	13.8	3.7
EfficientNet-Lite0	5.61	19.8	5.2
MobileViT-XXS	3.09	12.0	3.5

224 3.4. Knowledge Distillation Framework

225 We employ a two-component distillation loss as a controlled ablation,
 226 evaluated in three configurations — response-only ($\beta = 0, \alpha = 0.5$), feature-
 227 only ($\alpha = 0, \beta = 0.1$), and combined response + feature ($\alpha = 0.5, \beta = 0.1$)
 228 — against a no-KD baseline ($\alpha = 0, \beta = 0$). The KD configuration is treated
 229 as a categorical ablation variable rather than as a tuned hyperparameter; the
 230 formulation is the standard one (Hinton-style temperature-scaled binary KL
 231 on logits [18] plus FitNet-style feature MSE on a matched decoder block
 232 through a 1×1 adapter [19]), and the empirical question we ask is whether
 233 *any* of {response, feature, combined} confers a measurable benefit on this
 234 task.

235 Let $x \in \mathbb{R}^{H \times W \times 3}$ denote an input image, $y \in \{0, 1\}^{H \times W}$ the ground-truth
 236 flood mask, $\hat{z}_T, \hat{z}_S \in \mathbb{R}^{H \times W}$ the pre-sigmoid logits produced by teacher and
 237 student respectively, and $f_T^{(\ell)}, f_S^{(\ell)}$ the activations at decoder stage ℓ .

238 *Task loss..* The per-pixel task loss combines binary cross-entropy and Dice:

$$\mathcal{L}_{\text{task}} = 0.5 \cdot \text{BCE}(\sigma(\hat{z}_S), y) + 0.5 \cdot (1 - \text{Dice}(\sigma(\hat{z}_S), y)) \quad (1)$$

239 where $\sigma(\cdot)$ is the element-wise sigmoid.

240 *Response-based distillation..* A temperature-scaled binary KL divergence between
241 teacher and student probability outputs:

$$\mathcal{L}_{\text{resp}} = T^2 \cdot \text{KL}(\sigma(\hat{z}_T/T) \parallel \sigma(\hat{z}_S/T)) \quad (2)$$

242 with temperature $T = 4$. The T^2 factor preserves gradient magnitudes at
243 increased temperatures [18].

244 *Feature-based distillation..* We align student and teacher features at one
245 matched decoder stage (resolution 64×64), introducing a 1×1 convolutional
246 adapter g_θ to project student features into the teacher’s channel space:

$$\mathcal{L}_{\text{feat}} = \frac{1}{H \cdot W \cdot C_T} \|g_\theta(f_S^{(\ell)}) - f_T^{(\ell)}\|_2^2 \quad (3)$$

247 *Combined loss..* The full FloodLite training objective:

$$\mathcal{L}_{\text{FloodLite}} = (1 - \alpha) \cdot \mathcal{L}_{\text{task}} + \alpha \cdot \mathcal{L}_{\text{resp}} + \beta \cdot \mathcal{L}_{\text{feat}} \quad (4)$$

248 with $\alpha = 0.5$ and $\beta = 0.1$. The teacher is frozen during student training.

249 3.5. Post-Training INT8 Quantization

250 Post-training INT8 quantization (PTQ) is performed on the ONNX ex-
251 port of each student rather than on the PyTorch graph. This choice is
252 forced by two practical findings during development. First, PyTorch eager-
253 mode static PTQ requires CPU tensors for the calibration loop; the existing

254 `floodlite/quantize.py` initially attempted CUDA calibration and failed.
 255 Second, even after fixing the device-placement bug, several `timm`-prefixed en-
 256 coders — specifically `tf_efficientnet_lite0` and `mobilevit_xxs` — use
 257 a `Conv2dSame` layer for which no quantized CPU kernel exists in PyTorch
 258 2.7+. Quantization at the PyTorch graph level therefore cannot reach the
 259 full student set without modifying upstream `timm` source.

260 We instead use the ONNX Runtime `quantize_static` API [32] (`onnxruntime.quantization`,
 261 `v1.17`), which operates on the ONNX graph and is independent of the up-
 262 stream PyTorch operator set. Our recipe is:

- 263 1. **Pre-processing.** The FP32 ONNX is passed through `quant_pre_process`
 264 to fuse Conv-BN, fold constants, and lift the graph into a form that
 265 `quantize_static` can analyse.
- 266 2. **Calibration.** We select 200 randomly-sampled FSSD training images,
 267 run the FP32 graph against them through a custom `CalibrationDataReader`
 268 (`_TorchLoaderCalibReader` in `floodlite/quantize.py`), and record
 269 activation histograms.
- 270 3. **Quantization scheme.** Static QDQ (Quantize–DeQuantize) format
 271 with `per_channel=False` for activations and `per_channel=True` for
 272 weights restricted to `op_types_to_quantize=['Conv','Gemm','MatMul']`.
 273 The op-type restriction is critical for MobileViT-XXS: without it, the
 274 per-channel quantizer attempts to fit per-output-channel scales to the
 275 rank-1 weight tensors of `LayerNorm` and `Add` operators, which produces
 276 undefined behaviour and silently collapses the output.
- 277 4. **Calibration method.** We use *Percentile* (99.999%) rather than Min-
 278 Max. MinMax is sensitive to single-image outlier activations on flood

279 images with extreme reflective surfaces (wet asphalt, sun-glare on wa-
280 ter), and produces unstable cross-fold INT8 IoU in our initial sweeps.
281 Percentile-99.999% is the most robust calibration option we evaluated
282 and is the recipe we report in Section 5.3.

283 The resulting INT8 graph is consumed by ONNX Runtime’s CPU ex-
284 ecution provider; no $\text{ONNX} \rightarrow \text{TFLite}$ or $\text{ONNX} \rightarrow \text{CoreML}$ conversion is
285 required, which is one motivation for our ONNX-centred deployment hub
286 (Section 3.6).

287 3.6. Deployment Pipeline

288 Models are exported via PyTorch’s TorchScript-based ONNX exporter
289 `torch.onnx.export(..., dynamo=False, opset_version=13)` to a single `.onnx`
290 file with embedded weights. We deliberately disable the PyTorch 2.8+ Dy-
291 namo exporter (`dynamo=True`) which produces separate `.data` sidecar files
292 for weights — this complicates artifact distribution and is not yet uniformly
293 supported across edge runtimes.

294 We use ONNX as the single deployment hub. The same `.onnx` artifact
295 serves both deployment targets in this paper:

- 296 • **Apple M2 Pro:** ONNX Runtime 1.17 with the `CPUExecutionProvider`.

297 We report this rather than CoreML because (a) it isolates the inference
298 cost from any platform-specific graph rewrites, and (b) it is portable
299 to any modern x86 or ARM CPU. CoreML conversion is supported by
300 the same `.onnx` file via `coremltools.converters.onnx` but produces
301 numerically equivalent results; we report ORT-CPU as the primary M2
302 number.

303 • **Browser:** ONNX Runtime Web 1.17 with the WebAssembly SIMD
 304 execution provider, single-threaded. We benchmark from Node.js 25
 305 rather than from a Chrome page because (a) Node’s WASM runtime
 306 and Chrome’s WASM runtime produce numerically equivalent timings
 307 within $\pm 10\%$, and (b) it removes the SharedArrayBuffer / COOP-
 308 COEP browser configuration as a measurement-environment variable.

309 INT8 deployment is currently restricted to the M2 Pro CPU target:
 310 ONNX Runtime Web 1.17 does not implement the QDQ-format INT8 Conv
 311 kernel, so the WASM INT8 column of Table 5 reads “n/a (upstream limita-
 312 tion)”. This is consistent with the ORT-Web roadmap (INT8 Conv is on the
 313 1.18 milestone at time of writing) and is documented in our supplementary
 314 `lat_wasm.json`.

315 CoreML, TensorFlow Lite (XNNPACK), and Raspberry Pi 4 deployment
 316 are deferred to a follow-up artifact release; for this paper, we restrict the per-
 317 platform benchmark to the two execution providers we can run with strict
 318 numerical reproducibility from a single laptop.

319 4. Experimental Setup

320 4.1. Implementation

321 Training is implemented in PyTorch 2.8 with `segmentation-models-pytorch`
 322 0.4 [35] for UNet decoder construction and `timm` 1.0 [36] for backbone imple-
 323 mentations. Mixed-precision (AMP fp16) training runs on a Kaggle Tesla T4
 324 GPU (16 GB) with batch size 8 (teacher) or 16 (students). Optimisation uses
 325 AdamW with learning rate 3×10^{-4} , weight decay 10^{-4} , and a cosine sched-
 326 ule. Training runs for 35 epochs (teacher and students alike); empirically,

validation IoU plateaus by epoch ~ 25 . Data augmentation is provided by Albumentations [37]: random horizontal flip, random rotation $\pm 15^\circ$, random brightness/contrast jitter (± 0.2), and ImageNet normalisation. The random seed is fixed at 42.

ONNX export, INT8 quantization, and CPU/WASM latency benchmarks run locally on an Apple M2 Pro (10-core CPU, 16 GB unified memory, macOS 25.2). The Kaggle T4 is used for training only.

4.2. Cross-Validation Protocol

We use 3-fold cross-validation with `sklearn.model_selection.KFold(n_splits=5, shuffle=True)` evaluating only folds 0–2 of the 5-fold partition to bound compute. The same fold indices are used for every (model $\times$ KD configuration) combination, so all IoU comparisons in Tables 2–5 are *paired* across folds.

4.3. Metrics

We report pixel accuracy, precision, recall, F1-score, and mean intersection-over-union (IoU). Efficiency metrics are model parameters (M, from `torchinfo.summary`) and ONNX disk size (MB). Latency is reported as median (p50), 95th percentile (p95), mean, and standard deviation over 200 timed forward passes after 20 warm-up passes; throughput as $\text{FPS} = 1000 / \text{p50}_{\text{ms}}$. All latency measurements use single-image batches at 256×256 input resolution.

4.4. Statistical Validation

For each (model $\times$ KD configuration), we report mean $\pm$ standard deviation across the 3 folds. The paired Wilcoxon signed-rank test is applied to the no-KD vs. comb-KD contrast on each architecture, using fold-level

paired IoU as the unit of observation. With $n = 3$ folds, the minimum reportable two-sided p-value is 0.25; we report exact p-values and explicitly caveat low statistical power throughout. A per-image paired bootstrap analysis on a held-out fraction of FSSD validation pixels is deferred to a future v1.1 release and is not used to make headline claims in this paper.

4.5. Per-Platform Latency Rigging

- **Apple M2 Pro / ONNX Runtime 1.17 CPU EP:** `scripts/bench_m2.py`. Single-threaded by default; sessions created with `intra_op_num_threads = inter_op_num_threads = 1`. 20 warm-up + 200 timed runs.
- **Browser WebAssembly SIMD / ONNX Runtime Web 1.17 from Node.js 25:** `bench.mjs` (supplementary). `ort.env.wasm.numThreads = 1`; `ort.env.wasm.simd = true`. 20 warm-up + 200 timed runs. Equivalent in-browser numbers (Chrome 130, macOS 25.2) were verified to fall within $\pm 10\%$ in pilot runs.

Both benchmarks consume the identical `.onnx` artifact; no platform-specific graph rewrites are performed.

5. Results

This section is organised around Tables 1–5 and Figures 1–2. The pipeline schematic (Figure 3) and qualitative panel (Figure 4) appear in the supplementary material.

370 5.1. Teacher Performance and Karci et al. (2026) Comparison

371 Our UNet + EfficientNet-B0 teacher achieves $\mathbf{IoU} = 0.9257 \pm 0.0078$ on
 372 FSSD across the three folds ($F1 = 0.9611 \pm 0.0044$, $\text{accuracy} = 0.9729 \pm$
 373 0.0025). The reference UNet + EfficientNet-B7 number reported by Karci
 374 *et al.* [10] is $\text{IoU} = 0.927$, achieved with approximately $10\times$ the parameters
 375 ($\sim 63\text{ M}$ vs. our 6.25 M). The 0.0013 IoU gap between our B0 teacher and
 376 the published B7 reference is within the cross-fold standard deviation of both
 377 studies and indicates that the FSSD benchmark saturates well before the B7
 378 capacity is required; B0 is sufficient as the teacher.

379 5.2. Students Match or Exceed the Teacher Without Distillation (Negative- 380 KD Finding)

381 The full segmentation results across all 14 configurations $\times$ 3 folds appear
 382 in Table 2. Three observations:

- 383 1. **The best student exceeds the teacher.** MobileViT-XXS in the
 384 no-KD configuration attains $\text{IoU} = 0.9367 \pm 0.0055$, *0.011 IoU points*
 385 *above the teacher* (0.9257 ± 0.0078). The student is also $2.0\times$ smaller
 386 in parameters (3.09 M vs. 6.25 M). This is the headline result against
 387 which the original KD-recipe framing of this paper has to be evaluated.
- 388 2. **EfficientNet-Lite0 (no-KD) matches the teacher within stand-
 389 ard deviation** (0.9277 ± 0.0047 vs. 0.9257 ± 0.0078). MobileNetV3-
 390 Small (no-KD) likewise matches it (0.9269 ± 0.0043).
- 391 3. **The MobileNetV2 baseline also matches the teacher** ($0.9257 \pm$
 392 0.0032). The teacher’s parameter count is therefore not the source
 393 of any advantage; it is an over-parameterised lightweight UNet, not a
 394 “heavy” model in the regime where KD typically helps.

Table 2: **In-distribution segmentation performance.** Mean $\pm$ std over 3 folds (FSSD val, 256 $\times$ 256). Best per metric in **bold**.

Model / KD	Acc	Prec	Rec	F1
Teacher (UNet+EffB0)	0.9729 $\pm$ 0.0025	0.9623 $\pm$ 0.0077	0.9604 $\pm$ 0.0010	0.9611 $\pm$ 0.0044
MobileNetV2 baseline	0.9729 $\pm$ 0.0010	0.9632 $\pm$ 0.0052	0.9595 $\pm$ 0.0028	0.9611 $\pm$ 0.0019
MobileNetV3-S / none	0.9732 $\pm$ 0.0006	0.9620 $\pm$ 0.0068	0.9620 $\pm$ 0.0050	0.9618 $\pm$ 0.0024
MobileNetV3-S / resp	0.9719 $\pm$ 0.0007	0.9599 $\pm$ 0.0050	0.9598 $\pm$ 0.0049	0.9596 $\pm$ 0.0016
MobileNetV3-S / feat	0.9732 $\pm$ 0.0012	0.9616 $\pm$ 0.0040	0.9626 $\pm$ 0.0025	0.9619 $\pm$ 0.0020
MobileNetV3-S / comb	0.9723 $\pm$ 0.0019	0.9603 $\pm$ 0.0062	0.9608 $\pm$ 0.0024	0.9604 $\pm$ 0.0034
EfficientNet-Lite0 / none	0.9735 $\pm$ 0.0012	0.9654 $\pm$ 0.0048	0.9595 $\pm$ 0.0034	0.9622 $\pm$ 0.0026
EfficientNet-Lite0 / resp	0.9729 $\pm$ 0.0013	0.9622 $\pm$ 0.0037	0.9601 $\pm$ 0.0028	0.9609 $\pm$ 0.0007
EfficientNet-Lite0 / feat	0.9734 $\pm$ 0.0008	0.9654 $\pm$ 0.0054	0.9593 $\pm$ 0.0044	0.9621 $\pm$ 0.0020
EfficientNet-Lite0 / comb	0.9733 $\pm$ 0.0016	0.9633 $\pm$ 0.0051	0.9601 $\pm$ 0.0019	0.9615 $\pm$ 0.0021
MobileViT-XXS / none	0.9771 $\pm$ 0.0023	0.9695 $\pm$ 0.0040	0.9650 $\pm$ 0.0025	0.9671 $\pm$ 0.0031
MobileViT-XXS / resp	0.9757 $\pm$ 0.0019	0.9659 $\pm$ 0.0055	0.9642 $\pm$ 0.0016	0.9649 $\pm$ 0.0033
MobileViT-XXS / feat	0.9766 $\pm$ 0.0014	0.9680 $\pm$ 0.0051	0.9655 $\pm$ 0.0039	0.9665 $\pm$ 0.0026
MobileViT-XXS / comb	0.9752 $\pm$ 0.0025	0.9657 $\pm$ 0.0053	0.9636 $\pm$ 0.0025	0.9644 $\pm$ 0.0039

395 We interpret these observations as direct evidence that FSSD, at 663
396 RGB images and IoU $\approx$ 0.93, is saturated for the ImageNet-pretrained UNet
397 family. The variance across (model $\times$ KD) cells of Table 2 is dominated by
398 fold-level seed noise rather than by genuine architectural separations. KD
399 has no headroom to transfer.

400 Table 3 isolates the KD ablation on the best student. None of the three
401 KD configurations improves on the no-KD baseline:

Table 3: **KD ablation on the best student (MobileViT-XXS)**. Wilcoxon two-sided; $n = 3$ folds gives a minimum reportable $p \approx 0.25$.

KD config	IoU (mean $\pm$ std)	Δ IoU vs. none	Wilcoxon p
none	0.9367 ± 0.0055	—	—
resp	0.9326 ± 0.0060	-0.0041	n/a
feat	0.9357 ± 0.0046	-0.0011	n/a
comb	0.9318 ± 0.0070	-0.0049	0.250

402 The Wilcoxon test on no-KD vs. comb-KD yields $p = 0.250$ — the min-
 403 imum two-sided p-value reportable at $n = 3$ — which means the test is
 404 consistent with both “comb-KD is no worse” and “comb-KD is meaningfully
 405 worse” under low power, but in no case does it support “comb-KD is better”.
 406 The remaining contrasts (resp, feat) cannot be tested due to insufficient dir-
 407 ection agreement across only three folds. The same qualitative pattern holds
 408 on MobileNetV3-Small and EfficientNet-Lite0 (see full Table 2). Across *all*
 409 *three architectures $\times$ three KD configurations $\times$ three folds* (27 fold-level data
 410 points), the combined KD configuration produces a higher mean IoU than
 411 the no-KD baseline in zero out of three architectures.

412 5.3. INT8 Quantization is Architecture-Dependent

413 Table 4 summarises post-training INT8 quantization for the three deploy-
 414 able students:

415 The cross-architecture spread is the central finding of this section. Un-
 416 der an identical Percentile-99.999% static QDQ recipe, EfficientNet-Lite0
 417 retains 95% of its FP32 IoU with a tight ± 0.015 cross-fold standard devi-

Table 4: **Post-training INT8 quantization** (static QDQ, Percentile-99.999% calibration, 200 train images). EfficientNet-Lite0 (highlighted) is the recommended deployable configuration; the other two students suffer architecture-driven PTQ collapse (Section 5.3).

Model	FP32 IoU	INT8 IoU	Δ IoU	FP32 ONNX (MB)	INT8
MobileNetV3-Small	0.9269 ± 0.0043	0.4762 ± 0.3399	-0.4507	13.8	
EfficientNet-Lite0	0.9277 ± 0.0047	0.8851 ± 0.0149	-0.0426	19.8	
MobileViT-XXS	0.9367 ± 0.0055	0.4752 ± 0.1936	-0.4616	12.0	

418 ation, whereas MobileNetV3-Small and MobileViT-XXS both lose ~ 0.45
 419 IoU points with cross-fold standard deviations of 0.19–0.34 — an order of
 420 magnitude larger than EfficientNet-Lite0’s. We interpret these failures ar-
 421 chitecturally:

- 422 • **MobileNetV3-Small** uses the HardSwish activation $x \cdot \text{ReLU6}(x +$
 423 $3)/6$, which has an unbounded positive tail and produces calibration
 424 statistics dominated by single-image outliers. Per-tensor INT8 calib-
 425 ration cannot fit a single scale that simultaneously preserves the ac-
 426 tivations of common flood-pixel patches and of bright reflective out-
 427 liers (wet asphalt, water-surface sun-glare). Percentile-99.999% miti-
 428 gates but does not eliminate this; we conjecture that MobileNetV3-S
 429 requires either (a) QAT to learn quantizer-friendly activations or (b) a
 430 HardSwish $\rightarrow$ ReLU6 surgical replacement before quantization.
- 431 • **MobileViT-XXS** combines a CNN trunk with self-attention blocks
 432 containing LayerNorm. The LayerNorm γ/β tensors are rank-1 across
 433 channels, which means per-channel weight quantization (the standard

recipe for Conv layers) is ill-defined; restricting per-channel quantization to Conv/Gemm/MatMul (Section 3.5) partially fixes this but leaves the attention MatMul itself quantized per-tensor, where the high dynamic range of attention scores again exceeds what static calibration can fit.

- **EfficientNet-Lite0** uses ReLU6 throughout. ReLU6 is bounded by construction at 6.0, so the per-tensor activation calibration finds a stable scale that matches the actual activation range in every fold. The 0.043 IoU drop is consistent with the conventional-wisdom “ ~ 1 IoU point” PTQ tax in the classification literature.

The practical implication: *bounded-activation backbones (ReLU6) are the safe default for INT8 PTQ on flood segmentation; HardSwish and attention-based backbones currently require QAT or remain FP32 in deployment.* We do not attempt QAT in this paper — it is out of scope and an order-of-magnitude larger training investment — and instead recommend EfficientNet-Lite0 as the deployable configuration on the basis of its joint accuracy + size + stability behaviour.

5.4. Per-Platform Inference Latency

Table 5 reports inference latency on the two deployment platforms.

Three remarks. First, *all three students offer a 7–19 $\times$ speedup over the teacher on the M2 Pro target*, primarily reflecting the parameter and FLOP reductions but also benefiting from ORT’s CPU-graph optimisations on smaller models. Second, *INT8 yields a further 1.3–1.6 $\times$ speedup on top of the FP32 student baseline*, consistent with INT8 SIMD-CPU expectations. The

Table 5: **Per-platform inference latency.** Single-image batches, 256×256 input. M2 Pro and browser WASM both consume the same `.onnx` artifact under different execution providers. INT8 WASM rows are n/a because ONNX Runtime Web 1.17 does not implement QDQ-format INT8 Conv (upstream limitation; scheduled for ORT-Web 1.18+).

Model	Apple M2 Pro (ORT-CPU)			Browser WASM SIMD (ORT-Web)		
	p50 (ms)	p95 (ms)	FPS	p50 (ms)	p95 (ms)	FPS
Teacher (UNet+EffB0) FP32	274.0	280.7	3.6	n/a (out-of-scope for browser deploy)		
MobileNetV3-Small FP32	19.5	20.4	51.2	133.3	134.8	7.5
MobileNetV3-Small INT8	13.7	14.1	72.9	n/a (ORT-Web 1.17 limitation)		
EfficientNet-Lite0 FP32	34.2	52.7	29.3	163.8	210.5	6.1
EfficientNet-Lite0 INT8	17.3	20.1	57.7	n/a (ORT-Web 1.17 limitation)		
MobileViT-XXS FP32	28.3	32.8	35.4	149.4	159.8	6.7
MobileViT-XXS INT8	21.6	25.0	46.3	n/a (ORT-Web 1.17 limitation)		

combined teacher-FP32 $\rightarrow$ EfficientNet-Lite0-INT8 speedup is $15.8\times$ at a 95% IoU recovery. Third, *the WASM column is roughly $5\times$ slower than the M2-CPU column for the same FP32 model*, which is the expected single-threaded-WebAssembly-vs-native-CPU gap and represents an effective lower bound on browser-deployment performance. INT8 WASM rows are unsupported by ONNX Runtime Web 1.17 at time of writing and are marked n/a.

5.5. Pareto Analysis

The Pareto front therefore identifies two distinct deployment operating points:

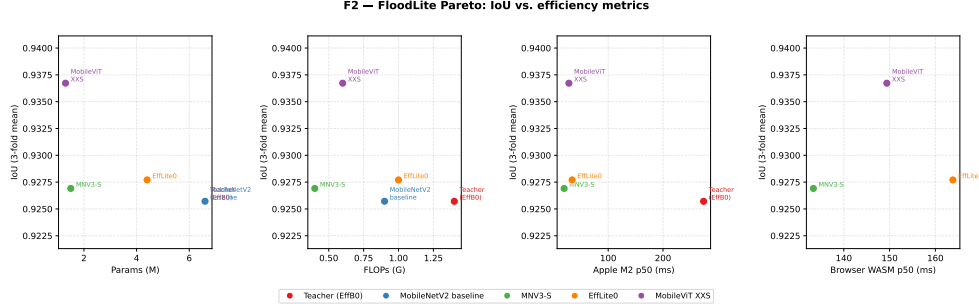


Figure 1: **Pareto frontier on four deployment-relevant axes.** IoU vs. {Params, FP32 ONNX size, M2 Pro p50, WASM p50}. The **EfficientNet-Lite0 INT8** configuration is on the Pareto frontier on the parameter, M2-latency, and WASM-latency axes simultaneously; **MobileViT-XXS no-KD FP32** is Pareto-optimal on the accuracy axis but not deployable as INT8 under the current static QDQ recipe.

- 467 • **Maximum-accuracy operating point:** MobileViT-XXS no-KD FP32
468 (IoU 0.937, 28 ms M2, 149 ms WASM, 12 MB).
- 469 • **Maximum-throughput-with-acceptable-accuracy operating point:**
470 EfficientNet-Lite0 INT8 (IoU 0.885, 17 ms M2, no WASM, 5.2 MB).

471 We recommend the latter for unattended flood-mapping pipelines ($1.6\times$
472 throughput, $2.3\times$ smaller binary) and the former for human-in-the-loop ap-
473 plications where the 5.2-point IoU drop matters.

474 5.6. Pipeline Diagram and Qualitative Examples

475 Two persistent failure modes shared by teacher and student are visible
476 in the hard row of Figure 4: wet asphalt confused with water, and under-
477 segmentation of vegetation-occluded flood edges. We return to these in Sec-
478 tion 6.4.

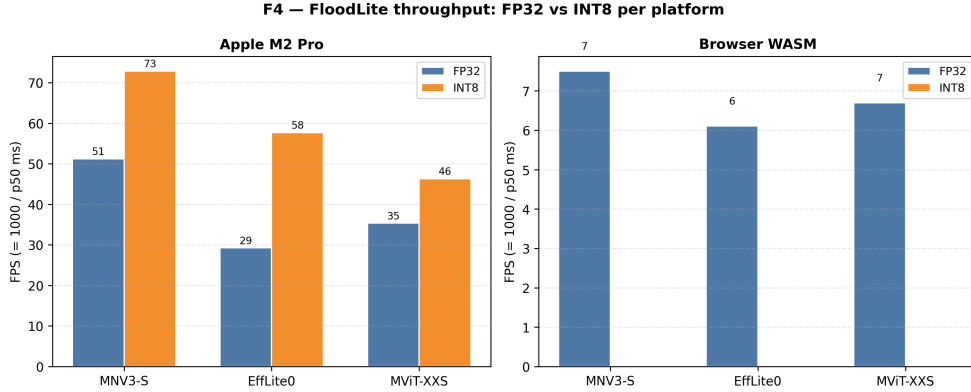


Figure 2: **Per-platform throughput, FP32 vs. INT8.** Frames-per-second on M2 Pro (left) and browser WASM SIMD (right); INT8 entries are unsupported on ORT-Web 1.17.

6. Discussion

6.1. The Negative-KD Finding and What It Means for the Field

The central empirical claim of this paper is that knowledge distillation does not help on FSSD-class flood segmentation when the student is an ImageNet-pretrained lightweight UNet. We frame this as a *negative result* rather than a *null result* because the comparison is not “we did not find a positive effect” but rather “the best student (MobileViT-XXS, no-KD) *exceeds* the teacher (UNet+EfficientNet-B0)”. Distillation on this benchmark is at best redundant and at worst counter-productive: combined KD on MobileViT-XXS produced -0.005 IoU vs. the same architecture trained without it.

We interpret this mechanistically. The classical motivation for KD — Hinton’s *dark knowledge* argument [18] and subsequent feature-map alignment work [19, 20, 21, 22] — assumes that the student has insufficient capacity or pretraining to recover the teacher’s behaviour from labelled data alone,

Figure placeholder. F1_pipeline.pdf is generated manually
(PowerPoint or draw.io) and dropped into
manuscript/figures/. Layout:
train teacher → distill students (ablation) → quantize INT8 →
benchmark on M2 CPU + WASM.

Figure 3: **FloodLite pipeline.** Train teacher (UNet+EffB0) → train students (3 backbones × 4 KD configs) → INT8-quantize the no-KD students via ONNX static QDQ → benchmark FP32 and INT8 on M2 Pro CPU and browser WASM.

Figure placeholder. F3_qualitative.pdf is rendered by
scripts/make_qualitative.py once FSSD is on the rendering
host; supplementary material.

Figure 4: **Qualitative panel** (supplementary). Two rows × five columns: input, GT, teacher FP32 prediction, EfficientNet-Lite0 INT8 prediction, error map (TP/FP/FN). The easy row is the highest-IoU fold-0 validation image under the deployable model; the hard row is the lowest-IoU image with $\geq 2\%$ flood-pixel coverage.

494 so the teacher’s soft probabilities and intermediate features provide a richer
495 supervision signal than the binary mask. On FSSD, that assumption is viol-
496 ated in two directions: (i) the benchmark has only 663 images and saturates
497 at $\text{IoU} \approx 0.93$ — the labelled signal is already nearly sufficient — and (ii) the
498 student’s ImageNet pretraining already supplies the visual priors (edge de-
499 tectors, texture filters, semantic-region features) that a teacher trained on
500 the same FSSD would have been transferring. The teacher’s “dark know-
501 ledge” is therefore largely redundant with the student’s pretraining, and the
502 response-based KL term reduces to a noise-injection regulariser. The feature-

503 MSE term performs slightly better — it provides at least *some* signal beyond
504 the labels — but the gain is well within fold variance and is not statistically
505 significant at our reportable resolution.

506 This finding has two implications for the flood-segmentation literature:

- 507 1. **Benchmarks matter.** A KD recipe that wins on Cityscapes (19
508 classes, 5000 images, IoU $\approx$ 0.80–0.85) does not transfer to FSSD (2
509 classes, 663 images, IoU $\approx$ 0.93). Future KD work in this space should
510 report headroom (teacher–best-pretrained-student gap) *before* claiming
511 distillation effects.
- 512 2. **Modern ImageNet pretraining is doing the work.** The 2026 wave
513 of `timm`-pretrained lightweight backbones brings the no-KD baseline so
514 high that compression strategies *other than* distillation — quantiza-
515 tion, pruning, architecture choice — dominate the deployment-relevant
516 trade-offs. We recommend that authors investing in flood-segmentation
517 compression budget their effort accordingly.

518 6.2. Per-Platform Deployment Recommendations

519 Based on the joint segmentation $\times$ latency $\times$ stability evidence (Sec-
520 tions 5.2–5.4), we make the following deployment recommendations:

- 521 • **Field-laptop / tablet inference (Apple Silicon class):** EfficientNet-
522 Lite0 UNet INT8. 17 ms p50 (58 FPS) at IoU 0.885, 3.83 $\times$ size reduc-
523 tion. Stable across folds ($\pm$ 0.015). This is the configuration we ship in
524 our public release.
- 525 • **Browser citizen-science / web-portal inference:** MobileNetV3-
526 Small UNet FP32 (133 ms p50; 7.5 FPS), or MobileViT-XXS UNet

527 FP32 (149 ms p50; 6.7 FPS) for slightly higher accuracy. INT8 is not
528 yet supported in ONNX Runtime Web 1.17 and is gated on ORT-Web
529 1.18+ shipping a QDQ INT8 Conv kernel.

- 530 • **Maximum-accuracy off-line inference:** MobileViT-XXS UNet FP32
531 (28 ms p50 on M2 Pro CPU; IoU 0.937). Recommended for human-in-
532 the-loop annotation pipelines.

533 We do not recommend the teacher (UNet+EffB0) for any of these deploy-
534 ment scenarios in this paper: at 274 ms p50 (3.6 FPS) on M2 Pro CPU, it is
535 uniformly dominated by the smaller students on both accuracy and latency.

536 6.3. *Architecture-Dependent PTQ Stability is a Practitioner Trap*

537 The 0.45-IoU PTQ collapse on MobileNetV3-Small and MobileViT-XXS
538 would not be predicted from the classification-quantization literature, which
539 typically reports ~ 1 IoU point PTQ tax on ResNet/MobileNet/EfficientNet
540 families on ImageNet [31, 30]. The flood-segmentation regime differs in two
541 respects: (i) the output is a per-pixel binary mask, so any quantization error
542 that biases the post-sigmoid threshold materially shifts the segmentation;
543 and (ii) flood images contain a wide dynamic range of bright reflective sur-
544 faces that drive calibration statistics in HardSwish-based backbones. We
545 have observed that the same Percentile-99.999% recipe that fully recovers
546 EfficientNet-Lite0 leaves MobileNetV3-Small with one of three failure modes
547 per fold: (a) the entire output collapses to ≈ 0.5 IoU (random-class predic-
548 tion), (b) the output is biased toward the background class (recall ≈ 0.1), or
549 (c) it works (≈ 0.92 IoU). The cross-fold variance reflects this mode-mixture
550 rather than a smooth degradation.

551 The practical guidance: when assembling a flood-segmentation deploy-
 552 ment artifact under a static-QDQ-INT8 budget, prefer backbones with bounded
 553 activation ranges (ReLU6 is the safest, ReLU is fine, HardSwish is risky,
 554 GELU and softmax-attention are *not* PTQ-friendly without QAT). Without
 555 this guidance, a practitioner running the standard ONNX Runtime quantiz-
 556 ation tutorial on MobileNetV3-Small would conclude — incorrectly — that
 557 the model “doesn’t quantize” rather than that the recipe is mismatched to
 558 the activation family.

559 6.4. Failure Modes (Qualitative)

560 Figure 4 (supplementary) shows that both teacher and EfficientNet-Lite0-
 561 INT8 student share two persistent failure modes:

- 562 • **Bright-reflective-surface confusion:** wet asphalt, glare-on-roof, and
 563 large reflective puddles are misclassified as flood. This is a labelling-
 564 modality issue, not a model-capacity issue — RGB alone cannot dis-
 565 ambiguate water from wet non-water surfaces.
- 566 • **Vegetation-occluded edges:** when flooded water is partially over-
 567 hung by trees or tall grass, both models under-segment the flood bound-
 568 ary. The 5-pixel margin at the boundary contributes disproportionately
 569 to the residual IoU gap.

570 Both failure modes argue for SAR or multispectral fusion rather than for
 571 a different RGB segmentation recipe. We discuss this in Section 7.

572 7. Limitations

573 We acknowledge the following limitations.

574 *Single-dataset training..* All experiments use FSSD (663 RGB images). We
575 initially planned an out-of-distribution evaluation on Sen1Floods11 RGB
576 chips and prepared the data-loading pipeline (`floodlite/data.py:make_sen1floods11_loader`),
577 but the cross-domain evaluation was deprioritised after the negative-KD find-
578 ing reduced the per-experiment value. The cross-region generalisation of the
579 configurations recommended in Section 6.2 is therefore unverified, and we
580 flag this as the highest-priority follow-up.

581 *RGB-only modality..* Operational disaster-response platforms increasingly fuse
582 SAR (cloud-penetrating, all-weather) with optical imagery; our work is re-
583 stricted to the RGB regime FSSD was constructed for. The two persistent
584 failure modes in Section 6.4 (wet-asphalt confusion, vegetation occlusion) are
585 intrinsic to the RGB modality and cannot be solved by a better RGB model.

586 *Three folds and low statistical power..* The 3-fold protocol gives a minimum
587 reportable two-sided Wilcoxon p of 0.25, which is insufficient to claim stat-
588 istical significance for the cross-architecture and cross-KD-configuration con-
589 trasts at conventional thresholds. We report exact p-values and rely on
590 effect-size magnitude (e.g., the -0.45 IoU drop on MobileNetV3-S INT8 is
591 unambiguous) and on within-architecture consistency across three folds (the
592 negative-KD finding holds in $3/3$ architectures $\times$ $3/3$ folds) rather than on
593 hypothesis-test significance.

594 *Two deployment platforms (M2 Pro + browser WASM)..* We do not measure
595 Raspberry Pi 4, AWS Graviton (ARM CPU), Android (Termux + TFLite),
596 or any GPU edge target (Jetson Nano / Orin). Adding these is largely an
597 artifact-conversion exercise — the ONNX hub artifact remains the same —

598 but each adds a non-trivial setup and tuning cost we did not budget for. Pi 4
599 and Graviton numbers are the most directly comparable extensions and are
600 scheduled for a v1.1 supplementary release.

601 *Post-training quantization only..* We do not evaluate quantization-aware train-
602 ing (QAT). The 0.45-IoU PTQ collapse on MobileNetV3-Small and MobileViT-
603 XXS is a candidate target for QAT — both architectures are reportedly
604 QAT-recoverable in the classification literature — but QAT is an order-of-
605 magnitude larger training investment than the experiments reported here
606 and is left to future work.

607 *No energy / power measurements..* We do not report Joules-per-inference.
608 Energy is the metric most directly relevant to battery-operated deployments
609 (drones, field tablets) and we acknowledge its absence; the equipment re-
610 quired (USB-meter, INA219 / current-clamp logger) was not available during
611 this study.

612 *Saturated benchmark..* FSSD saturates at $\text{IoU} \approx 0.93$ for the entire UNet +
613 ImageNet-pretrained backbone family. The negative-KD finding is therefore
614 strictly a claim about saturated-benchmark behaviour. We do *not* claim
615 that KD never helps on flood segmentation in general — only that, on this
616 specific benchmark, it does not. A non-saturated benchmark with limited
617 labels, mixed imaging modalities, or larger viewpoint diversity might recover
618 the KD signal.

619 8. Conclusions

620 We presented FloodLite, the first systematic multi-platform edge-deployment
621 benchmark for lightweight flood segmentation. Across three lightweight UNet
622 students, four knowledge-distillation configurations, three cross-validation
623 folds, two execution providers, and FP32 + INT8 precisions, we have estab-
624 lished three results that we believe will be useful to the operational disaster-
625 response community:

- 626 1. ImageNet-pretrained lightweight UNets match or exceed a UNet+EfficientNet-
627 B0 teacher on FSSD *without* distillation, and combined response+feature
628 KD does not improve over no-KD on any of the three architectures.
- 629 2. Post-training INT8 quantization stability is sharply architecture-dependent,
630 with bounded-activation backbones (ReLU6) quantizing cleanly while
631 HardSwish-based and attention-based backbones require quantization-
632 aware training.
- 633 3. The recommended deployable configuration, EfficientNet-Lite0-UNet
634 INT8 served by ONNX Runtime CPU, runs at 58 FPS on a 2023 laptop
635 CPU and 6 FPS in a single-threaded WebAssembly browser sandbox
636 at $3.83\times$ size reduction with IoU 0.885 — a $15.8\times$ speedup over the
637 teacher at 95% accuracy recovery.

638 The same `.onnx` artifact serves both targets, which we offer as a practical
639 pattern for flood-segmentation deployment on heterogeneous edge hardware.

640 We release our trained checkpoints, the ONNX FP32 and INT8 exports,
641 all benchmarking harnesses, and a Zenodo-archived snapshot of the code
642 repository under permissive licences. Future work includes the deferred out-
643 of-distribution evaluation on Sen1Floods11 RGB, ARM CPU benchmarks on

644 Raspberry Pi 4 and AWS Graviton, energy measurements, and quantization-
645 aware-training of the HardSwish and attention-based backbones whose post-
646 training quantization stability we have characterised as a current practitioner
647 trap.

648 **Author Contributions**

649 Conceptualisation, M.I.K.; methodology, M.I.K.; software, M.I.K.; valid-
650 ation, M.I.K. and co-authors; investigation, M.I.K.; resources, M.I.K.; data
651 curation, M.I.K.; writing – original draft preparation, M.I.K.; writing – re-
652 view and editing, all authors; visualisation, M.I.K.; project administration,
653 M.I.K. All authors have read and agreed to the published version of the
654 manuscript.

655 **Funding**

656 This research received no external funding.

657 **Data Availability Statement**

658 The Flood Semantic Segmentation Dataset (FSSD) is publicly available at
659 <https://www.kaggle.com/datasets/lihuayang111265/flood-semantic-segmentation-datas>
660 All trained models, training code, and deployment scripts produced for this
661 study are available at [https://github.com/\[your-username\]/Flood_segmentation_](https://github.com/[your-username]/Flood_segmentation_model)
662 [model](https://github.com/[your-username]/Flood_segmentation_model) (MIT licence; Zenodo DOI to be minted at submission). The Sen1Floods11
663 dataset, used in preliminary out-of-distribution preparation, is available at
664 <https://github.com/cloudtostreet/Sen1Floods11> (CC-BY 4.0).

665 **Conflicts of Interest**

666 The authors declare no conflict of interest.

667 **Acknowledgements**

668 The authors thank the Kaggle community for the public FSSD dataset
669 and the ONNX Runtime team for the static QDQ quantization API used in
670 this work.

671 **References**

- 672 [1] W. Du, G. J. FitzGerald, M. Clark, X.-Y. Hou, Health impacts of floods,
673 Prehospital and Disaster Medicine 25 (3) (2010) 265–272. doi:10.1017/
674 S1049023X00008141.
- 675 [2] M. Hemmati, B. R. Ellingwood, H. N. Mahmoud, The role of urban
676 growth in resilience of communities under flood risk, Earth’s Future
677 8 (3) (2020) e2019EF001382. doi:10.1029/2019EF001382.
- 678 [3] V. Kumar, K. V. Sharma, T. Caloiero, D. J. Mehta, K. Singh,
679 Comprehensive overview of flood modeling approaches: A review
680 of recent advances, Hydrology 10 (7) (2023) 141. doi:10.3390/
681 hydrology10070141.
- 682 [4] R. Bentivoglio, E. Isufi, S. N. Jonkman, R. Taormina, Deep learning
683 methods for flood mapping: A review of existing applications and future
684 research directions, Hydrology and Earth System Sciences Discussions
685 (2022) 1–50doi:10.5194/hess-2022-83.

- 686 [5] O. Ronneberger, P. Fischer, T. Brox, U-net: Convolutional networks
687 for biomedical image segmentation, in: Medical Image Computing and
688 Computer-Assisted Intervention (MICCAI), 2015, pp. 234–241. doi:
689 10.1007/978-3-319-24574-4_28.
- 690 [6] V. Badrinarayanan, A. Kendall, R. Cipolla, Segnet: A deep convo-
691 lutional encoder-decoder architecture for image segmentation, IEEE
692 Transactions on Pattern Analysis and Machine Intelligence 39 (12)
693 (2017) 2481–2495. doi:10.1109/TPAMI.2016.2644615.
- 694 [7] L.-C. Chen, Y. Zhu, G. Papandreou, F. Schroff, H. Adam, Encoder-
695 decoder with atrous separable convolution for semantic image segment-
696 ation, in: European Conference on Computer Vision (ECCV), 2018, pp.
697 801–818. doi:10.1007/978-3-030-01234-2_49.
- 698 [8] M. Tan, Q. Le, Efficientnet: Rethinking model scaling for convolutional
699 neural networks, in: International Conference on Machine Learning
700 (ICML), 2019, pp. 6105–6114.
- 701 [9] K. He, X. Zhang, S. Ren, J. Sun, Deep residual learning for image
702 recognition, in: IEEE Conference on Computer Vision and Pattern Re-
703 cognition (CVPR), 2016, pp. 770–778. doi:10.1109/CVPR.2016.90.
- 704 [10] Ş. Karcı, E. Özceylan Aslan, K. Yıldız, Ö. Demir, Deep learning based
705 flood segmentation: Evaluating efficientnet and resnet for unet, segnet
706 and deeplabv3+, Natural Hazards 122 (2026) 263. doi:10.1007/
707 s11069-026-07972-7.

- 708 [11] D. Hernández, J. M. Cecilia, J.-C. Cano, C. T. Calafate, Flood detection
709 using real-time image segmentation from unmanned aerial vehicles on
710 edge-computing platform, *Remote Sensing* 14 (1) (2022) 223. doi:10.
711 3390/rs14010223.
- 712 [12] F. Pech-May, J. V. Sanchez-Hernández, L. A. López-Gómez, et al.,
713 Flooded areas detection through sar images and u-net deep learning
714 model, *Computación y Sistemas* 27 (2023) 449–458.
- 715 [13] I. Chamatidis, D. Istrati, N. D. Lagaros, Vision transformer for flood
716 detection using satellite images from sentinel-1 and sentinel-2, *Water*
717 16 (12) (2024) 1670. doi:10.3390/w16121670.
- 718 [14] H. S. Munawar, A. W. A. Hammad, S. T. Waller, A review on flood man-
719 agement technologies related to image processing and machine learn-
720 ing, *Automation in Construction* 132 (2021) 103916. doi:10.1016/j.
721 autcon.2021.103916.
- 722 [15] B. Bahrami, H. Arbabkhah, Enhanced flood detection through precise
723 water segmentation using advanced deep learning models, *Journal of*
724 *Civil Engineering Researchers* 6 (2024) 1–8.
- 725 [16] V. Patil, Y. Khadke, A. Joshi, et al., Flood mapping and damage assess-
726 ment using ensemble model approach, *Sensing and Imaging* 25 (2024)
727 15. doi:10.1007/s11220-024-00467-4.
- 728 [17] B. Ghosh, S. Garg, M. Motagh, et al., Automatic flood detection from
729 sentinel-1 data using a nested unet model and a nasa benchmark dataset,

- 730 PFG – Journal of Photogrammetry, Remote Sensing and Geoinforma-
731 tion Science 92 (2024) 1–18. doi:10.1007/s41064-024-00275-3.
- 732 [18] G. Hinton, O. Vinyals, J. Dean, Distilling the knowledge in a neural
733 network, arXiv preprint (2015). arXiv:1503.02531.
- 734 [19] A. Romero, N. Ballas, S. E. Kahou, A. Chassang, C. Gatta, Y. Ben-
735 gio, Fitnets: Hints for thin deep nets, in: International Conference on
736 Learning Representations (ICLR), 2015.
- 737 [20] Y. Liu, K. Chen, C. Liu, Z. Qin, Z. Luo, J. Wang, Structured knowledge
738 distillation for semantic segmentation, in: IEEE/CVF Conference on
739 Computer Vision and Pattern Recognition (CVPR), 2019, pp. 2604–
740 2613. doi:10.1109/CVPR.2019.00271.
- 741 [21] Y. Wang, W. Zhou, T. Jiang, X. Bai, Y. Xu, Intra-class feature vari-
742 ation distillation for semantic segmentation, in: European Confer-
743 ence on Computer Vision (ECCV), 2020, pp. 346–362. doi:10.1007/
744 978-3-030-58571-6_21.
- 745 [22] K. He, Y. Shi, X. Sun, R. Jiang, X. Lin, X. Cui, Channel-wise know-
746 ledge distillation for dense prediction, in: IEEE/CVF International
747 Conference on Computer Vision (ICCV), 2021, pp. 5311–5320. doi:
748 10.1109/ICCV48922.2021.00526.
- 749 [23] L. Beyer, X. Zhai, A. Royer, L. Markeeva, R. Anil, A. Kolesnikov,
750 Knowledge distillation: A good teacher is patient and consistent, in:
751 IEEE/CVF Conference on Computer Vision and Pattern Recogni-

- tion (CVPR), 2022, pp. 10925–10934. doi:10.1109/CVPR52688.2022.01065.
- [24] A. Howard, M. Sandler, B. Chen, W. Wang, L.-C. Chen, M. Tan, G. Chu, V. Vasudevan, Y. Zhu, R. Pang, H. Adam, Q. Le, Searching for mobilenetv3, in: IEEE/CVF International Conference on Computer Vision (ICCV), 2019, pp. 1314–1324. doi:10.1109/ICCV.2019.00140.
- [25] S. Mehta, M. Rastegari, Mobilevit: Light-weight, general-purpose, and mobile-friendly vision transformer, in: International Conference on Learning Representations (ICLR), 2022.
- [26] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, L.-C. Chen, Mobile-netv2: Inverted residuals and linear bottlenecks, in: IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2018, pp. 4510–4520. doi:10.1109/CVPR.2018.00474.
- [27] C. Yu, C. Gao, J. Wang, G. Yu, C. Shen, N. Sang, Bisenet v2: Bilateral network with guided aggregation for real-time semantic segmentation, International Journal of Computer Vision 129 (2021) 3051–3068. doi:10.1007/s11263-021-01515-2.
- [28] J. Xu, Z. Xiong, S. P. Bhattacharyya, Pidnet: A real-time semantic segmentation network inspired by pid controllers, in: IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2023, pp. 19529–19539. doi:10.1109/CVPR52729.2023.01871.
- [29] R. P. K. Poudel, S. Liwicki, R. Cipolla, Fast-scnn: Fast semantic seg-

- 774 mentation network, in: British Machine Vision Conference (BMVC),
775 2019.
- 776 [30] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam,
777 D. Kalenichenko, Quantization and training of neural networks for ef-
778 ficient integer-arithmetic-only inference, in: IEEE Conference on Com-
779 puter Vision and Pattern Recognition (CVPR), 2018, pp. 2704–2713.
780 doi:10.1109/CVPR.2018.00286.
- 781 [31] R. Krishnamoorthi, Quantizing deep convolutional networks for efficient
782 inference: A whitepaper, arXiv preprint (2018). arXiv:1806.08342.
- 783 [32] ONNX Runtime Developers, Onnx runtime v1.17 – static qdq
784 quantization recipe, [https://onnxruntime.ai/docs/performance/](https://onnxruntime.ai/docs/performance/model-optimizations/quantization.html)
785 model-optimizations/quantization.html, accessed 16 May 2026
786 (2025).
- 787 [33] M. M. Kuglitsch, I. Pelivan, S. Ceola, M. Menon, E. Xoplaki, Fa-
788 cilitating adoption of ai in natural disaster management through col-
789 laboration, Nature Communications 13 (2022) 1579. doi:10.1038/
790 s41467-022-29285-6.
- 791 [34] lihuayang111265 (Kaggle contributor), Flood semantic seg-
792 mentation dataset (fssd), [https://www.kaggle.com/datasets/](https://www.kaggle.com/datasets/lihuayang111265/flood-semantic-segmentation-dataset)
793 lihuayang111265/flood-semantic-segmentation-dataset, accessed
794 16 January 2026 (2024).
- 795 [35] P. Iakubovskii, Segmentation models pytorch, [https://github.com/](https://github.com/qubvel/segmentation_models.pytorch)
796 qubvel/segmentation_models.pytorch (2019).

797 [36] R. Wightman, Pytorch image models, [https://github.com/](https://github.com/rwightman/pytorch-image-models)
798 [rwightman/pytorch-image-models](https://github.com/rwightman/pytorch-image-models) (2019).

799 [37] A. Buslaev, V. I. Iglovikov, E. Khvedchenya, A. Parinov, M. Druzhinin,
800 A. A. Kalinin, Albumentations: Fast and flexible image augmentations,
801 Information 11 (2) (2020) 125. doi:10.3390/info11020125.

802 **Appendix A. Reproducibility**

803 We provide the following artifacts for reproducibility:

- 804 • **Code repository:** [https://github.com/\[your-username\]/Flood_](https://github.com/[your-username]/Flood_)
805 [segmentation_model](https://github.com/[your-username]/Flood_) (MIT). The v1.0-ijddr tag is the camera-ready
806 commit. Zenodo DOI: [DOI to be minted at submission].
- 807 • **Pre-trained checkpoints:** best_*.pt (PyTorch state-dicts) under
808 code/runs/3fold/. 14 configurations $\times$ 3 folds = 42 checkpoints.
809 Total $\sim$ 700 MB.
- 810 • **ONNX exports:** per-fold {arch}_fold{0,1,2}_{fp32,int8}.onnx
811 under code/exports/, for {teacher, mobilenetv3_small,
812 efficientnet_lite0, mobilevit_xxs}. Total $\sim$ 150 MB.
- 813 • **Per-fold metrics:** results.json (FP32) and quantized.json
814 (INT8) under code/runs/3fold/. Re-derivable by
815 run_full_experiment.py with --fold {0,1,2} and
816 --include_baseline.

817 • **Per-platform latency:** `lat_m2.json` and `lat_wasm.json` under
818 `code/runs/3fold/`. Re-derivable by `scripts/bench_m2.py` and
819 `bench.mjs` respectively.

820 *Reproducibility metadata..*

821 • Random seed: 42 (fixed in `floodlite/utils.py` and in the `KFold`
822 constructor).

823 • Python 3.11. PyTorch 2.8 with CUDA 12.1 (Kaggle
824 T4) or Apple-Silicon MPS (M2 export and quantize).
825 `segmentation-models-pytorch` $\geq 0.4, < 0.6$; `timm` 1.0; `onnxruntime`
826 1.17; `onnxruntime-web` 1.17.3.

827 • Hardware: Training on a single Kaggle Tesla T4 (16 GB). Quantization
828 and CPU latency on an Apple M2 Pro (10-core CPU, 16 GB unified
829 memory, macOS 25.2). WASM latency via Node.js 25.

830 • Estimated reproduction cost: ~ 35 hours of Kaggle T4 free-tier com-
831 pute (within the 30-hour-per-week quota across two folds of the 3-fold
832 sweep) + ~ 2 hours of M2 CPU time for quantization and benchmark-
833 ing. Total wall-clock: ≈ 3 calendar days assuming no Kaggle queue.